

รายงานโครงงานฉบับสมบูรณ์ (Final Project Report)

0214321 Web Application Design and Development Module | ภาคเรียนที่ 1/2569 | เอกสารประกอบการนำเสนอ
Final Project (25%)

- กำหนดส่ง: ก่อนวันนำเสนอ Week 15 (วันจันทร์ที่ 15 กันยายน 2569) — ส่ง 1 ฉบับต่อทีม พร้อมกับ Deliverables อื่นๆ ตาม final_project_brief.docx
- เอกสารนี้เป็นรายงานฉบับสมบูรณ์ — คนละไฟล์กับ README.md ในตัว repo (README เน้นวิธีติดตั้ง/รันโปรเจกต์ ส่วนรายงานเน้นภาพรวม การออกแบบ และการสะท้อนผลการเรียนรู้)
- ใช้ประกอบการนำเสนอและ Demo ในวัน Final Presentation — อาจารย์และเพื่อนทีมอื่นจะอ่านคู่กับการดูระบบทำงานจริง
- รวม Reflection รายบุคคลของสมาชิกทุกคนไว้ในส่วนท้ายของรายงานฉบับนี้แล้ว

ส่วนที่ 1 — ข้อมูลทีมและโครงงาน

ชื่อโครงงาน:

LmSound

ชื่อทีม:

LmSound

สมาชิกในทีม:

ลำดับ	ชื่อ-นามสกุล	รหัสนิสิต	บทบาทหลัก
1.	นายศุภกร ยางสมบูรณ์	6720210102	ออกแบบฟังก์ชันฝั่งผู้ใช้ และ ออกแบบ UI หน้า home และธีมมืด
2.	นายอานนท์ ศรีรัฐ	6720210057	ออกแบบฟังก์ชันฝั่งแอดมิน และทำ ระบบแผนที่ จัดทำฐานข้อมูล

GitHub Repository:

https://github.com/amonsrirat/lm_sound

Production URL (Vercel):

<https://lmsound.arnoncore.com/>

ส่วนที่ 2 — ภาพรวมโครงการ

ปัญหา / ที่มาของโครงการ:

ผู้ใช้ที่ต้องการอ่านหนังสือหรือทำงานมักต้องทดลองเดินทางไปหลายสถานที่โดยไม่มีข้อมูลกลางที่บอกระดับเสียง บรรยากาศ ภาพจริง เสียงตัวอย่าง และสิ่งอำนวยความสะดวก เช่น Wi-Fi แสงสว่าง ปลั๊กไฟ แอร์ หรือพัดลม อีกทั้งการเปิดเสียงเพื่อช่วยโฟกัสก็ต้องสลับไปมาระหว่างหลายแอป

ทีมจึงพัฒนา LMSound ให้เป็นพื้นที่กลางสำหรับค้นหาจุดอ่านหนังสือ ดูตำแหน่งจากแผนที่ ฟังเสียงบรรยากาศ และจัดเพลงผ่อนคลาย โดยออกแบบให้ผู้ดูแลระบบจัดการสถานที่ สื่อ รูปภาพ เพลง เพลย์ลิสต์ และธีมเว็บไซต์ได้โดยไม่ต้องแก้ไขโค้ด

กลุ่มผู้ใช้เป้าหมาย และคุณค่าที่ระบบนี้มอบให้:

กลุ่มผู้ใช้	คุณค่าที่ได้รับ
นิสิต นักเรียน และนักอ่าน	ค้นหาจุดอ่านหนังสือที่เหมาะสมกับระดับเสียงและช่วงเวลา พร้อมดูแผนที่และภาพจริง
ผู้ทำงานหรือผู้ต้องการโฟกัส	เปิดเสียงบรรยากาศ ปรับ mixer ตั้งเวลา และเลื่อนตำแหน่งเพลงได้ในหน้าเดียว
ผู้ใช้มือถือ	ใช้งานผ่าน responsive UI และติดตั้งเป็น PWA
ผู้ดูแลระบบ	จัดการสถานที่ สื่อ เพลง เพลย์ลิสต์ สิ่งอำนวยความสะดวก และธีม

ส่วนที่ 3 — ฟีเจอร์หลักที่พัฒนา

ระบุฟีเจอร์ทั้งหมดที่ทำเสร็จจริง

ฟีเจอร์	คำอธิบายสั้นๆ	สถานะ
Spot และแผนที่	สร้าง แก้ไข ลบ ค้นหา พร้อมพิกัด ภาพ เสียง ระดับเสียง ช่วงเวลา	เสร็จ
	และหมด	
แผนที่แบบ Modal	เห็นหมดทั้งหมด หมดที่เลือกไฮไลต์สีฟ้า และการดัดแปลงรายการ	เสร็จ
สิ่งอำนวยความสะดวก	ไอคอน Wi-Fi แสงสว่าง ปลั๊กไฟ แอร์ พัดลม และรายการอื่น	เสร็จ
Audio Engine	เล่น หยุด เปลี่ยนเพลง volume seek mixer focus recipes	เสร็จ
	และ sleep timer	
คลังเพลงผ่อนคลาย	เพลง เพลย์ลิสต์ การเผยแพร่ การให้คะแนน	เสร็จ
	และล็อกเนื้อหาสำหรับผู้ไม่ login	
Admin Content Management	จัดการรูป เสียง เพลง playlist และสถานะเผยแพร่	เสร็จ
Authentication	สมัคร ยืนยันอีเมล ลืมรหัสผ่าน รีเซต และ Google Login	เสร็จ
Responsive และ PWA	Bottom navigation โปรไฟล์มือถือ install prompt และ	เสร็จ
	service worker	
Theme และเทศกาล	light/dark โลโก้ favicon และช่วงวันเวลา theme จาก admin	เสร็จ
Upload ที่เสถียร	บีบอัดรูป direct upload R2 progress retry	เสร็จ
	และไฟล์เสียงขนาดใหญ่	
ฟีเจอร์	คำอธิบายสั้นๆ	สถานะ

ส่วนที่ 4 — การออกแบบระบบฉบับสมบูรณ์ (System Design)

โครงสร้างสถาปัตยกรรม (Controller-Service-Model) และเทคโนโลยีหลักที่ใช้: [Next.js/Prisma/PostgreSQL/Vercel](#) + [ไลบรารีเพิ่มเติม](#)

ระบบใช้แนวคิด Controller-Service-Model ในรูปแบบ Next.js App Router: Route Handler ทำหน้าที่เป็น controller รับ request และตรวจสอบสิทธิ์, action/service รวม business logic และ validation, Prisma Client เป็น data access layer และ PostgreSQL เป็นฐานข้อมูลหลัก ส่วนหน้าใช้ React Client Components และ Context สำหรับ audio engine ขณะที่ไฟล์ binary ถูกส่งตรงจาก browser ไป Cloudflare R2 ผ่าน presigned URL

ชั้นระบบ	เทคโนโลยี / หน้าที่
Presentation	Next.js App Router, React 19, TypeScript, Tailwind CSS 4, Leaflet / React Leaflet
Application	Route Handlers, Server Actions, Zod, JWT session, bcryptjs
Data	Prisma Client, PostgreSQL 16, relational models และ migration

ชั้นระบบ	เทคโนโลยี / หน้าที่
Object Storage	Cloudflare R2 ผ่าน AWS SDK S3-compatible API และ presigned PUT URL
Deployment	Docker multi-stage, Docker Compose, Docker Hub และ VPS หลัง reverse proxy
External Services	Google OAuth, Resend, OpenStreetMap และ Google Drive integration

โครงสร้างข้อมูลสุดท้าย (Resource/Model และความสัมพันธ์):

Model	ความสัมพันธ์และหน้าที่
User	เจ้าของ Spot, ratings, favorites และ auth tokens; มี role USER/ADMIN
Spot	สถานที่อ่านหนังสือ มีพิกัด ภาพ เสียง ระดับเสียง เวลาแนะนำ amenities และ author
MediaAsset	metadata เช่น fileKey, mimeType, size, url, folder, isPublished และสถิติ
RelaxationPlaylist / Track	เพลย์ลิสต์และตารางเชื่อมเพลงพร้อม position
TrackRating / SpotRating / Favorite	คะแนนและรายการถูกใจแบบหนึ่งรายการต่อผู้ใช้ต่อรายการ
SiteSetting	key-value สำหรับโลโก้ แบนเนอร์ อิม ข้อความ และช่วงเวลาเทศกาล
AuthToken / GoogleDriveConnection	token สำหรับ email flow และข้อมูล OAuth ของ Google Drive

API Endpoints ทั้งหมด:

Method	Path	ต้อง Login?	คำอธิบาย
GET	/api/auth/me	ไม่บังคับ	ตรวจสอบ session และผู้ใช้ปัจจุบัน
POST	/api/auth/register	ไม่บังคับ	สมัครสมาชิกและยืนยันอีเมล
POST	/api/auth/login	ไม่บังคับ	เข้าสู่ระบบด้วย username/email
POST	/api/auth/logout	ไม่บังคับ	ล้าง session cookie
GET	/api/auth/google และ callback	ไม่บังคับ	Google OAuth
POST	/api/auth/verify-email	ไม่บังคับ	ยืนยันรหัสอีเมล
POST	/api/auth/password-reset	ไม่บังคับ	ขอรหัสและรีเซ็ตรหัสผ่าน
GET/POST	/api/spots	GET ไม่บังคับ / POST login	อ่านและสร้างสถานที่
GET/PATCH/DELETE	/api/spots/[id]	แก้ไขต้อง login	ดู แก้ไข หรือลบตามสิทธิ์
POST	/api/spots/[id]/engagement	ต้อง login	favorite และ rating
GET/POST	/api/relaxation	ให้คะแนนต้อง login	เพลง เพลย์ลิสต์ การฟัง และ rating
GET/POST/DELETE/PATCH	/api/admin/relaxation	ADMIN	จัดการเพลงและ playlist
GET/POST/PATCH/DELETE	/api/admin/media	ADMIN	คลังไฟล์และ metadata
GET/DELETE	/api/admin/media/[fileId]	ADMIN	ดูหรือลบไฟล์
POST	/api/uploads/presign	ต้อง login	ขอ presigned URL
POST	/api/uploads/complete	ต้อง login	ตรวจสอบ object และบันทึก metadata
GET	/api/uploads/status	ต้อง login	ตรวจสอบสถานะอัปโหลด

Method	Path	ต้อง Login?	คำอธิบาย
GET	/api/uploads/[...key]	ตามนโยบายไฟล์	เสิร์ฟไฟล์ที่อนุญาต

จุด Authorization ที่ทำจริง (ใครทำอะไรได้บ้าง):

ผู้ใช้ทั่วไปดูสถานที่และเพลงที่เผยแพร่ได้ แต่ต้อง login ก่อนฟังเพลงออนไลน์ ให้คะแนน กดถูกใจ หรือสร้าง/แก้ไขข้อมูล

เจ้าของสถานที่เท่านั้นที่แก้ไขหรือลบ Spot ของตนเองได้ ส่วน ADMIN จัดการ Spot และสื่อได้ทั้งหมด

Route ของ admin ใช้ requireAdmin และ action สำคัญใช้ requireAuth ก่อนแตะฐานข้อมูล

Server เป็นผู้สร้าง fileKey และตรวจ userId, MIME type, size และ object ใน R2 ไม่เชื่อ path หรือ owner จาก client

ส่วนที่ 5 — Security & Technical Quality

ยืนยันมาตรการที่ทำจริงในระบบ (ตามที่เรียนใน Week 10)

☒	รหัสผ่านเข้ารหัสด้วย bcrypt ไม่เก็บเป็น plain text
☒	ใช้ Prisma Client ในการ query ทั้งหมด (ไม่มี raw SQL ที่เสี่ยง SQL Injection)
☒	ป้องกัน XSS — ไม่ใช้ dangerouslySetInnerHTML กับข้อมูลที่ผู้ใช้กรอกโดยไม่ sanitize
☒	ตรวจสอบข้อมูลนำเข้าด้วย Zod schema ก่อนบันทึกลงฐานข้อมูล
☒	มี Authorization check ป้องกันผู้ใช้แก้ไข/ลบข้อมูลของคนอื่น

ส่วนที่ 6 — Git Workflow & Deployment

สรุป Branching/PR Workflow ที่ทีมใช้ตลอดโปรเจกต์:

ทีมใช้ main เป็น branch production และแยกงานใน anon กับ Ninja สมาชิก push งานของตนเองขึ้น origin แล้วหัวหน้าทีม merge เข้า main ก่อน deploy ขั้นตอน production คือ build Docker multi-stage, tag image ตาม commit และ latest, push Docker Hub แล้วให้ VPS pull image ใหม่และ recreate เฉพาะ app container โดยคง PostgreSQL และ volume uploads ไว้

ขั้นตอน	สิ่งที่ทำจริง
Development	พัฒนาบน branch anon หรือ Ninja
Integration	merge งานเข้า main และ push origin/main
Build	สร้าง image arnonsrirat/lmsound ด้วย Dockerfile multi-stage
Registry	push tag commit และ latest ไป Docker Hub
VPS	docker compose pull และ up -d --no-deps lmsound
Verification	ตรวจ logs และ curl production ให้ได้ HTTP 200

ปัญหาที่เจอระหว่าง Deploy และวิธีแก้ (ถ้ามี):

ปัญหาระหว่าง deploy ได้แก่ database connection หลุดและพื้นที่ดิสก์ VPS ลดลงจาก Docker image/logs จึงตรวจ connection, cleanup image/logs และดูแล database container อย่างระมัดระวัง ส่วนการอัปเดตมือถือแก้ด้วย compression, queue/status, retry และ direct upload ไป R2

ส่วนที่ 7 — การทดสอบระบบ (Testing)

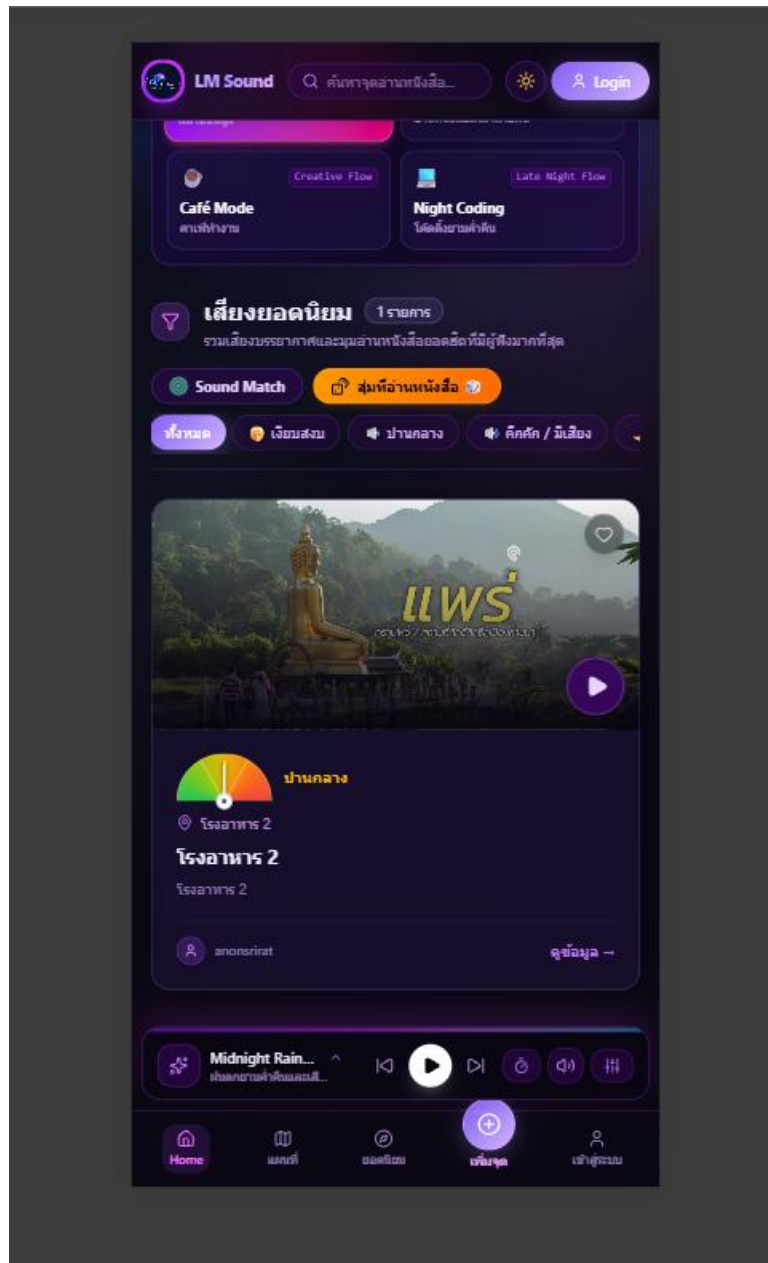
วิธีที่ทีมทดสอบระบบก่อนนำเสนอ (manual test cases หลักๆ ที่ตรวจแล้ว):

กรณีทดสอบ	วิธีตรวจ	ผล
สมัคร/ยืนยันอีเมล	สมัคร รับรหัส และยืนยัน	ผ่าน
Login/Logout	เข้าสู่ระบบและตรวจ session	ผ่าน
Google Login	ทดสอบ OAuth callback และ production redirect	ผ่านเมื่อ env ถูกต้อง
ลิ้มรสผ่าน	ขอรหัส Resend และตั้งใหม่	ผ่านเมื่อ env พร้อม
Spot CRUD	สร้าง แก้ไข ลบ และ owner guard	ผ่าน
แผนที่	มือถือ/desktop เห็นหมดและเลือกหมด	ผ่าน
Audio	เล่น หยุด เปลี่ยน volume/mixer/seek	ผ่าน
Upload	รูปหลายไฟล์และเสียงใหญ่พร้อม progress	ผ่านหลังเพิ่ม R2 flow
Admin	media เพลง playlist theme Drive	ผ่าน
Build/Deploy	tsc build Docker push pull HTTP health	ผ่าน

ส่วนที่ 8 — Screenshot หน้าจอหลักของระบบจริง

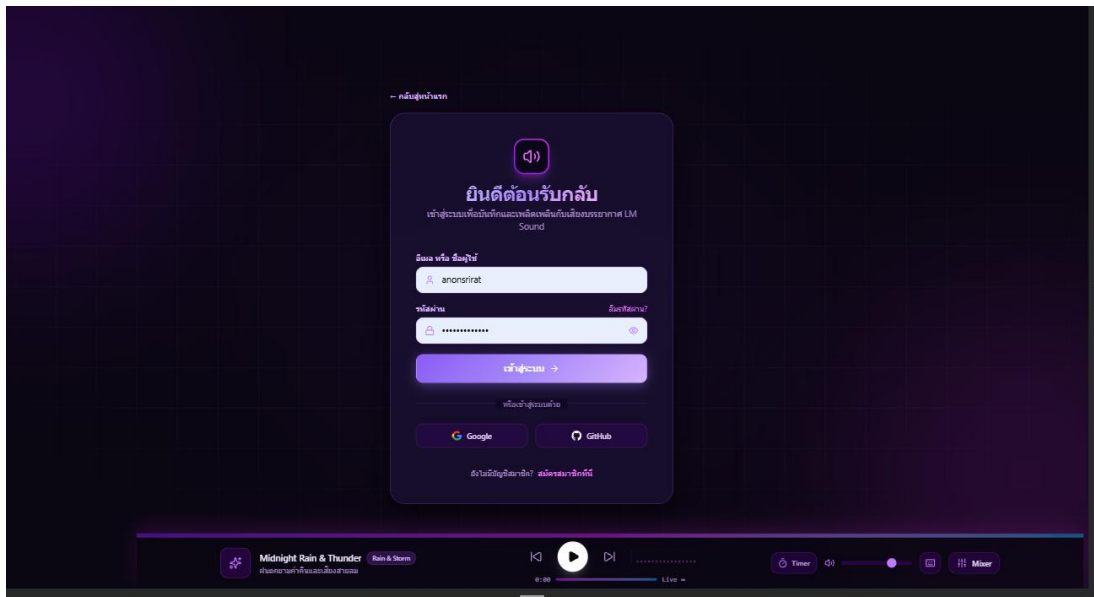
แนบภาพหน้าจอของระบบที่ทำงานได้จริงแล้ว (ไม่ใช่ wireframe) อย่างน้อย 3 หน้า

หน้าจอที่ 1 หน้า Home และรายการสถานที่



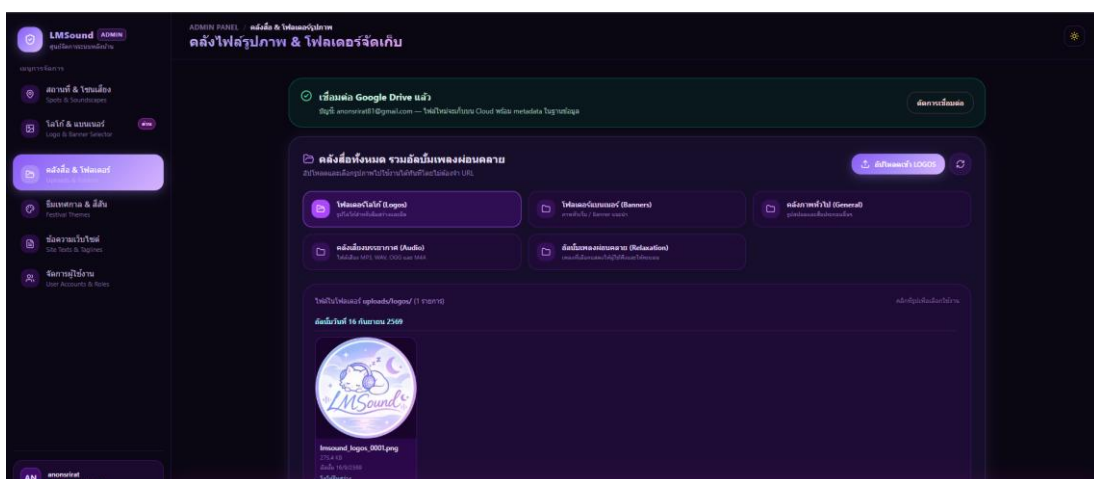
หน้าแรกแสดง banner, soundscape, filter, การ์ดสถานที่ และ audio player

หน้าจอที่ 2 หน้า Login และ Authorization



หน้า login พร้อม password และ provider ภายนอก

หน้าจอที่ 3 Admin Media Library



หน้า admin สำหรับจัดการคลังรูปภาพ ไฟล์เสียง และสื่อ

ส่วนที่ 9 — ปัญหาที่พบและบทเรียนที่ได้

ปัญหา/อุปสรรคสำคัญที่เจอตลอดโปรเจกต์ และวิธีที่ทีมแก้ไข:

ฐานข้อมูลหรือ Prisma เคยมีปัญหาจนข้อมูลสถานะที่ไม่แสดง ทีมจึงตรวจ connection, migration, container และแยก fallback ออกจาก production data

การโหลดรูปและอัปโหลดหลายไฟล์ทำให้มือถือค้าง จึงใช้ lazy loading, compression, queue/status, retry และ direct upload ไป R2

แผนที่ใน modal เคยเลื่อนหรือแสดงผลผิด จึงเพิ่ม invalidateSize, bounds, marker ทุกจุด, double-click สำหรับปัดหมุน และจัดการใหม่

เสียงและ modal เคยทับกันบนมือถือ จึงใช้ portal, z-index ชัดเจน, ซ่อน player เมื่อเปิด editor/profile และใช้ toast แทน window.alert

ธีม light/dark เคยอ่านยาก จึงกำหนด palette, logo light/dark, favicon sync และตั้งค่าจาก admin

สิ่งที่เรียนรู้ในภาพรวม และสิ่งที่จะพัฒนาต่อถ้ามีเวลาเพิ่ม:

ทีมได้เรียนรู้การออกแบบ full-stack ตั้งแต่ requirement, responsive UI, relational data modeling, authorization, OAuth, email workflow, object storage, Docker และ deployment จริง การแก้ปัญหาจากมือถือทำให้เห็นความสำคัญของ performance budget, network behavior และ progressive enhancement

หากมีเวลาเพิ่ม ควรเพิ่ม automated end-to-end tests, monitoring/error tracing, multipart resumable upload, image CDN transformation, offline cache และระบบแก้ไข profile/email แบบยืนยันอีเมลครบวงจร

ส่วนที่ 10 — Reflection รายบุคคล

สมาชิกทุกคนกรอกส่วนของตัวเอง — ใช้ประกอบการประเมิน Individual Reflection & Contribution (5 คะแนน)

สมาชิกคนที่ 1 — ชื่อ-นามสกุล: _____ นายอานนท์ ศรีรัฐ _____ รหัสนิสิต: __6720210057__

บทบาท/สิ่งที่รับผิดชอบหลักในโปรเจกต์:

ออกแบบฟังก์ชันฝั่งแอดมิน และทำระบบแผนที่ จัดทำฐานข้อมูล

สิ่งที่ได้เรียนรู้ / ทักษะที่พัฒนาขึ้นระหว่างทำโปรเจกต์นี้:

ได้เรียนรู้การใช้งาน github การทำงานร่วมกันกับเพื่อนในกลุ่ม การแบ่งงานกัน และการแก้ปัญหาด้วยกัน ส่วนการทำงานก็ได้เรียนรู้การจัดเก็บไฟล์ในรูปแบบใหม่

ความท้าทายส่วนตัวที่เจอ และวิธีที่จัดการ/แก้ปัญหา:

ตอนที่ทำระบบแอดมินการจัดเก็บรูปภาพ มันเต็มเครื่องเลยเปลี่ยนไปใช้ Google Drive แต่พอใช้ไปปุ๊บทำให้เว็บดึงข้อมูลหนักมาก ส่งผลต่อไปยังผู้ใช้ ทำให้ผู้ใช้งานทางฝั่งมือถือนำไปไม่ได้เลย จึงต้องมีการเปลี่ยนไปใช้ การจัดเก็บแบบ cloudflare R2 ทำให้การจัดเก็บรวดเร็วขึ้น ระบบจึงมีความเสถียรมากขึ้น

สมาชิกคนที่ 2 — ชื่อ-นามสกุล: ____นายศุภกร ยางสมบูรณ์____ รหัสนิสิต: _6720210102____

บทบาท/สิ่งที่รับผิดชอบหลักในโปรเจกต์:

ออกแบบฟังก์ชันฝั่งผู้ใช้ และออกแบบ UI หน้า home และอิมมิต

สิ่งที่ได้เรียนรู้ / ทักษะที่พัฒนาขึ้นระหว่างทำโปรเจกต์นี้:

ได้เรียนรู้การทำงานเป็นทีม และการใช้ github

ได้รู้เกี่ยวกับการเขียนเว็บ และ การใช้ Vps กับ docker และการตั้งค่า domain

ความท้าทายส่วนตัวที่เจอ และวิธีที่จัดการ/แก้ปัญหา:

ได้ลองแก้ไข ui ของเว็บไซต์และการเขียนฟังก์ชันต่างๆของuser ว่าทำยังไงให้ไม่ error ด้วยเมื่อรวมกับของเพื่อน และปัญหาที่เจอคือหน้า Ui ของ อิมไม่สอดคล้องกับหน้า Admin เกิดจากการที่ผมสื่อสารไม่ดีและไม่ได้เข้าไปเช็คหน้า Admin ว่ามีอะไรบ้าง

ส่วนที่ 11 — ตรวจสอบ Deliverables ก่อนส่ง

☒	ลิงก์ Production Deployment บน Vercel ที่ใช้งานได้จริง
☒	ลิงก์ GitHub Repository ที่มีประวัติ branch / Pull Request / Code Review ครบถ้วน
☒	README.md อธิบายโปรเจกต์ พี่เจอร์หลัก และวิธีติดตั้งรันบนเครื่อง (แยกจากรายงานฉบับนี้)
☒	สไลด์นำเสนอสั้นๆ สำหรับใช้ตอน Demo Day
☒	รายงานฉบับสมบูรณ์นี้ กรอกรครบทุกส่วน รวม Reflection รายบุคคลของทุกคนแล้ว

ลงชื่อสมาชิกทุกคน (ยืนยันว่าได้ตรวจสอบเนื้อหารายงานนี้ร่วมกัน):

สมาชิกคนที่ 1

สมาชิกคนที่ 2

อานนท์

ศุภกร

วันที่ส่ง:

17 กันยายน 2569